Storage Types (Block & Object)

Ceph: A unified storage system that provides Block (RBD), Object (RADOS Gateway), and File (CephFS) storage from a single cluster.

MinIO: A high-performance, S3-compatible Object store often used for private cloud deployments.

Storage Roles in E-commerce

MinIO (Object Storage): Used for Unstructured Data. In e-commerce, this includes product high-resolution images, video demonstrations, and customer-generated content like review photos.

Ceph (Block Storage): Used for Structured Data and performance-critical applications. In this scenario, Ceph provides the "virtual hard drives" (RBD) that host the actual databases for orders, users, and inventory.

2. Setting up Required files

- Create a folder and name it with the SRN
- Extract the shared zip file inside the folder created in step 1

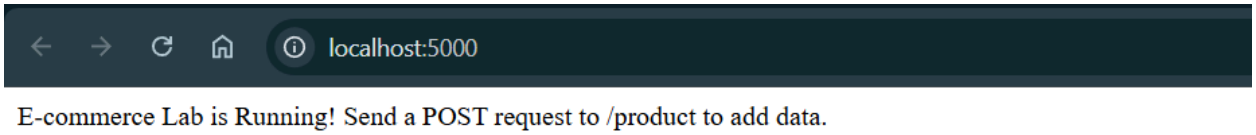
```
51    try:
52        # 2. Object Storage Action (MinIO)
53        bucket = "cs111"
54        if not minio_client.bucket_exists(bucket):
55            minio_client.make_bucket(bucket)
```

-
- Go to line 53 and rename the bucket name with your SRN for minIO Server UI to create the bucket with your SRN.

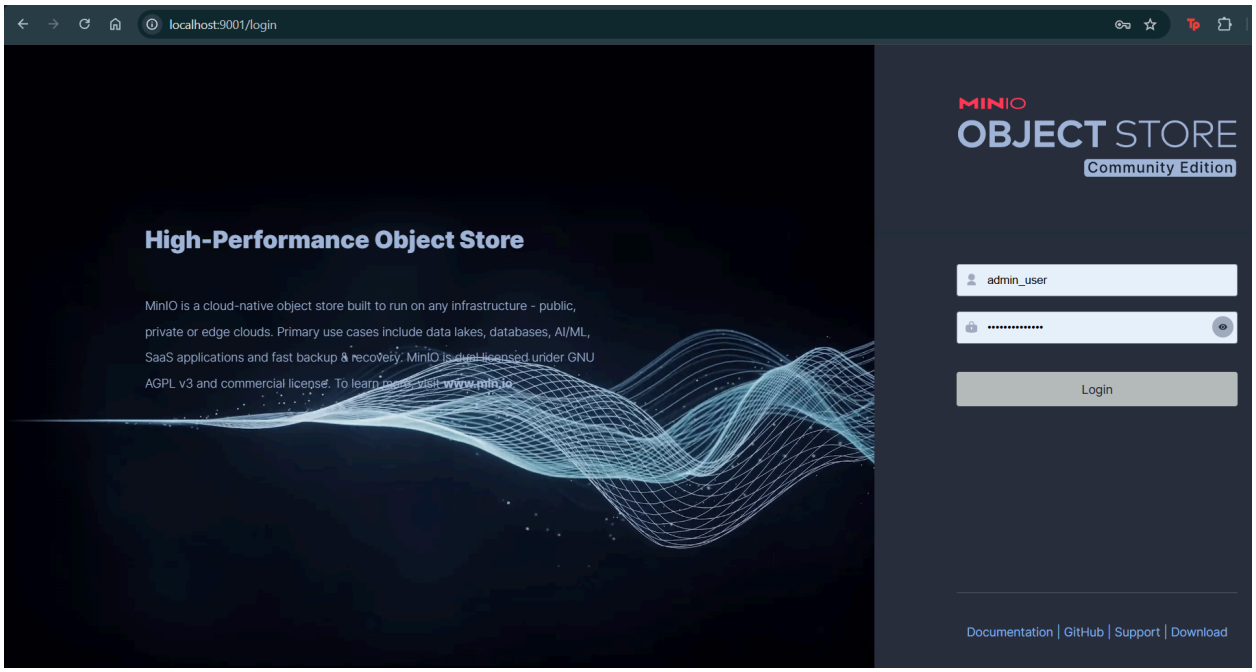
2. Setup Steps for Students

- Build and Start: Open a terminal in the <SRN folder> and run:
- docker-compose up --build
- Verify Services:
- Flask App: http://localhost:5000
- MinIO Dashboard: http://localhost:9001 (User/Pass: minioadmin)

Screenshot-1 browser http://localhost:500

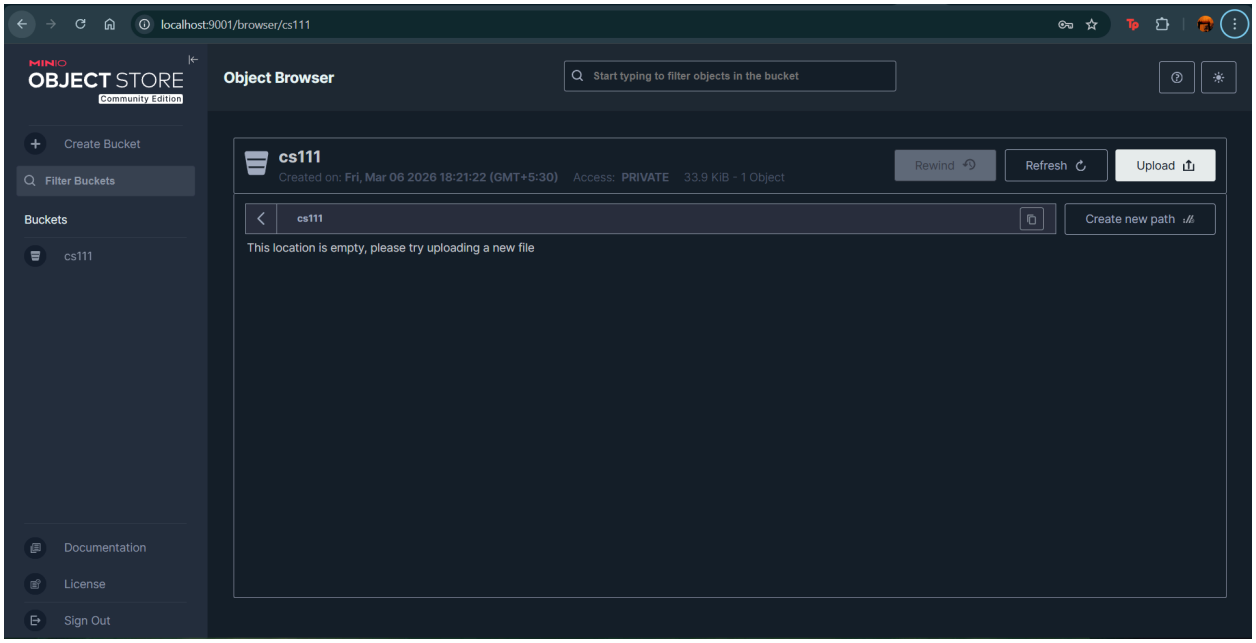


Screenshot-2 Browser: MinIO Dashboard: http://localhost:9001



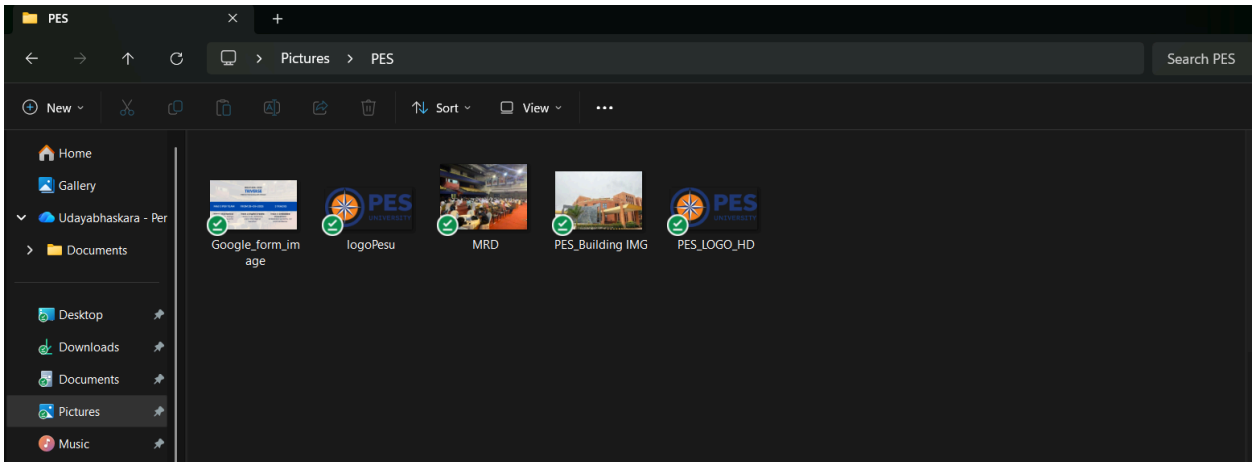
Username and password can be found in docker-compose.yml file line 9 and 10, loginto minIO Dashboard

Screenshot-3 Browser: MinIO Dashboard



Observe the bucket created with the given SRN

Uploading media to the minIO and ceph



Go to one of the folder where you have media files stored, preferably images

- open the terminal and follow execute the curl command
    - Upload Data: Use curl to add a product:
    - curl -X POST -F "name=Phone" -F "price=500" -F "image=@your_image.jpg" http://localhost:5000/product
    - If one is using powrshell replace ‘curl’ with ‘curl.exe’ in the command
        - curl.exe -X POST -F "name=Phone" -F "price=500" -F "image=@your_image.jpg" http://localhost:5000/product

You should get the response from the server as below

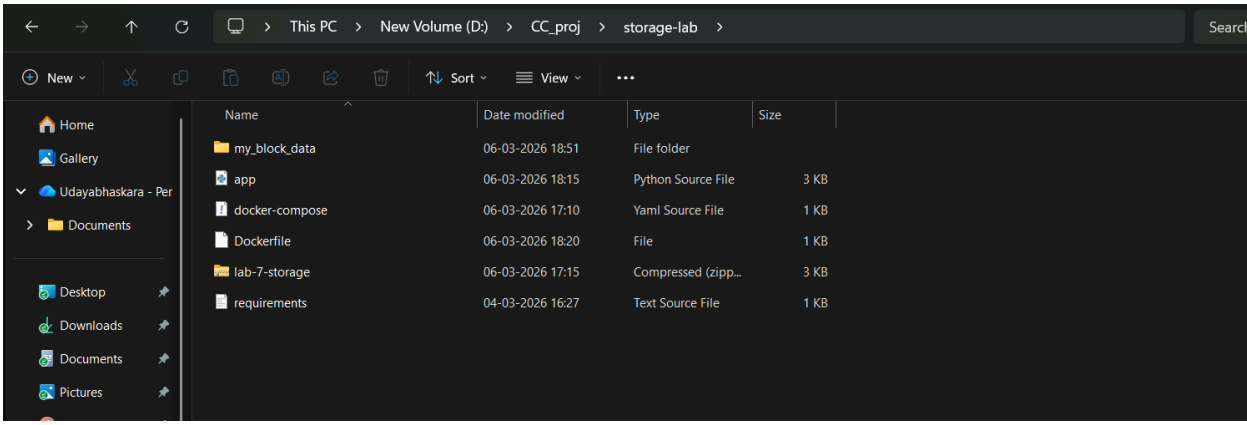
```
PS C:\Users\91990\OneDrive\Pictures\PES> curl.exe -X POST -F "name=Phone" -F "price=500" -F "image=@logoPesu.png" http:/
/localhost:5000/product
{"msg":"Structured data in Block, Image in Object","status":"success"}
PS C:\Users\91990\OneDrive\Pictures\PES>
```

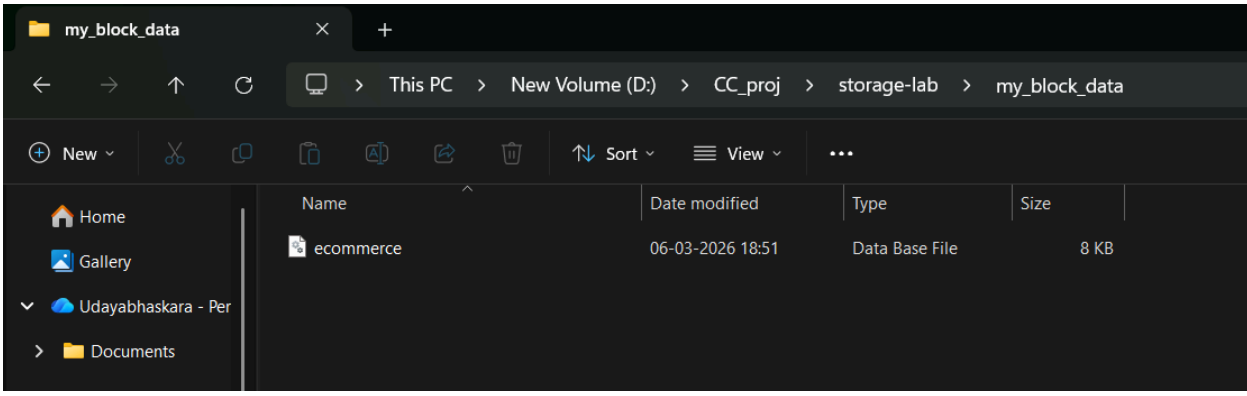
3. The 4 Laboratory Experiments

Experiment 1: Identifying the Access Method

Action: Look into the local folder storage-lab/my_block_data. You will see ecommerce.db. Now, try to find your_image.jpg in that same folder.

Here the project folder is named storage-lab, go to your project folder {folder created with your SRN} and find my_block_data.





Observation: The DB file is visible on the host OS. The image is missing from the local file system; **it only exists inside the MinIO dashboard**.

Concept: Block Storage presents as a local drive; Object Storage is abstracted behind an API.

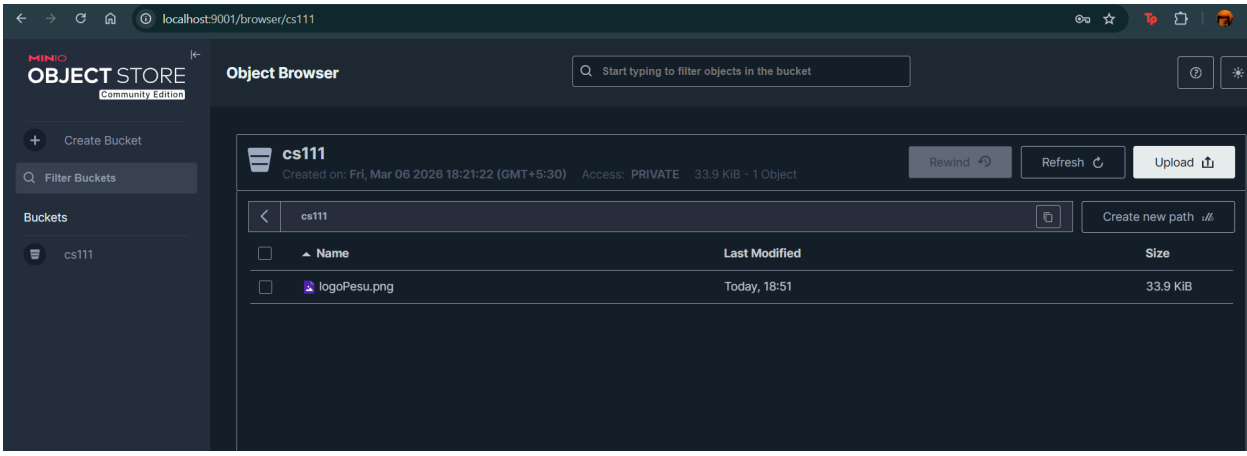
To state it more clearly, Ceph provides a block storage but we won't be able to view the data locally since its encoded through DBs used(we can use queries to check the details of the data stored)
Run the below command and observe(only structure of the data is available through query:

```
connect('/mnt/block_volume/ecommerce.db'); cursor = conn.cursor(); cursor.execute('SELECT * FROM product'); print(curso
.fetchall()); conn.close()"
[(1, 'Phone', 500.0, 'logoPesu.png')]
PS C:\Users\91990\OneDrive\Pictures\PES>
```

docker exec -it storage-lab-webapp-1 python -c "import sqlite3; conn = sqlite3.connect('/mnt/block_volume/ecommerce.db'); cursor = conn.cursor(); cursor.execute('SELECT * FROM product'); print(cursor.fetchall()); conn.close()"

In case of minIO, the same file uploaded can be previewed.

The same can be checked using minIO Dashboard.



Click on the **image** and preview the image.

Experiment 2: Metadata vs. File Headers

Action: In app.py, change the upload lines under **def add_product()**

app.py > add_product

```python
@app.route('/')
def home():
    return "E-commerce Lab is Running! Send a POST request to /p

@app.route('/product', methods=['POST'])
def add_product():
    name = request.form.get('name')
    price = request.form.get('price')
    image = request.files.get('image')
    # 1. Define your metadata dictionary
    # Values must be strings
    file_metadata = {
        "x-amz-meta-product-name": str(name),
        "x-amz-meta-product-price": str(price)
    }
    if not image:
        return jsonify({"error": "No image uploaded"}), 400
```

```python
# 1. Define your metadata dictionary
    # Values must be strings
    file_metadata = {
        "x-amz-meta-product-name": str(name),
        "x-amz-meta-product-price": str(price)
    }
```

And in the same app.py

Modify

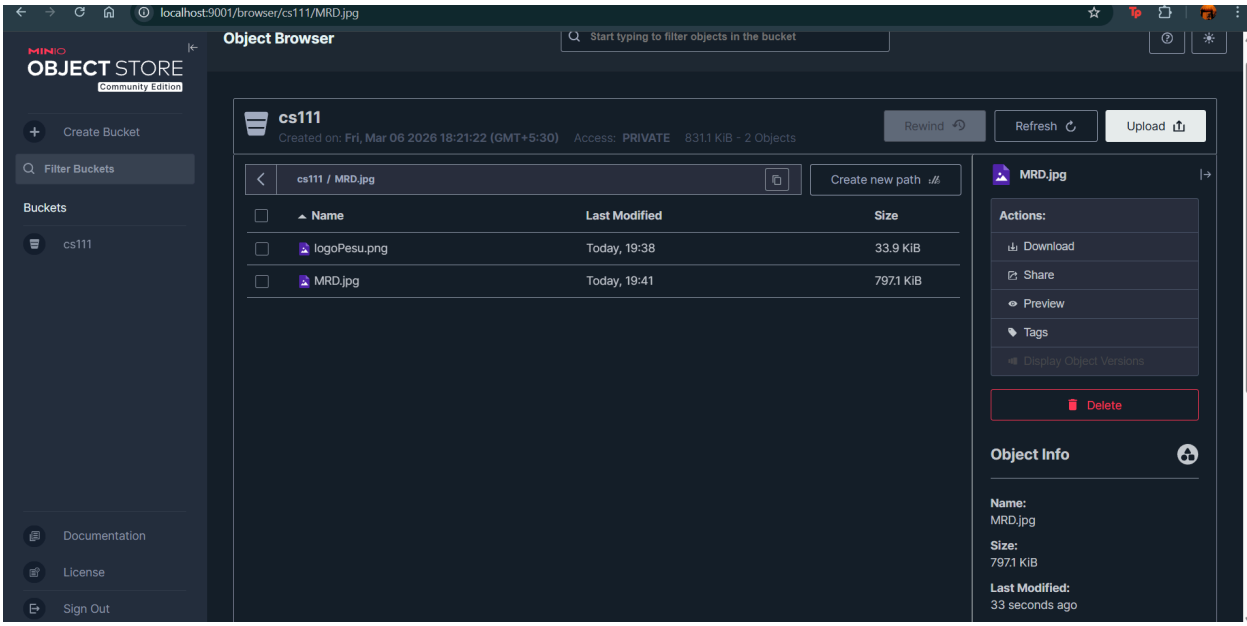
```python
minio_client.fput_object(bucket, image.filename, temp_path)
```

To

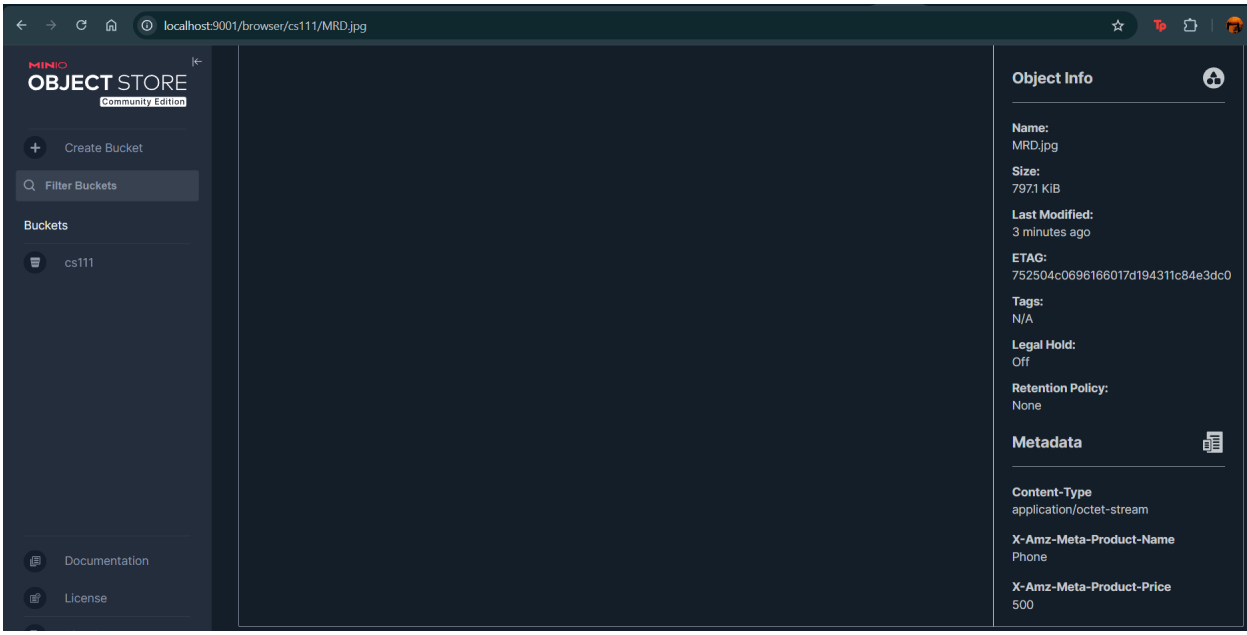
```python
minio_client.fput_object(
        bucket,
        image.filename,
        temp_path,
        metadata=file_metadata  # <--- Added this line
    )
```

upload another image using the same curl command and check the file "metadata" in the MinIO UI.

curl -X POST -F "name=Phone" -F "price=500" -F "image=@another_image.jpg"
here , the other image uploaded is MRD.jpg



Observation: You can see "product-Name" and “Product-Price” in the UI without opening the file. But the same isn’t possible with the .db file in your folder.



Aspect Changed: Enrichment of data with custom tags.

Concept: Object Stores support rich metadata; Block Stores only know "size" and "location."

To verify execute the command to check the structure of DB:

```
docker exec -it storage-lab-webapp-1 python -c "import sqlite3; conn = sqlite3.connect('/mnt/block_volume/ecommerce.db'); cursor = conn.cursor(); cursor.execute('SELECT * FROM product'); print(cursor.fetchall()); conn.close()"
```

These two experiments are for the students to Try

Experiment 3: Scaling & Fragmentation

Action: Open docker-compose.yml and change the volume mapping to a small, fixed-size virtual disk (advanced) or simply fill the my_block_data folder with dummy files until the disk is full.
Observation: The Flask app will throw a Database Full or Disk Error. Meanwhile, MinIO will continue accepting images as long as the Docker host has space.
Aspect Changed: Storage capacity limits.
Concept: Object Storage scales horizontally (add more nodes); Block Storage is constrained by the volume size.

Experiment 4: Recovery & High Availability

Action: Run docker-compose stop minio. Attempt to upload a product. Then run docker-compose start minio and docker-compose stop webapp.
Observation: When MinIO is down, the DB still works but images fail. When webapp is down, your data in my_block_data remains persistent and can be attached to a new container immediately.
Aspect Changed: Service uptime/availability.
Concept: Separation of Concerns. Data (Storage) lives independently of the Logic (Microservice).